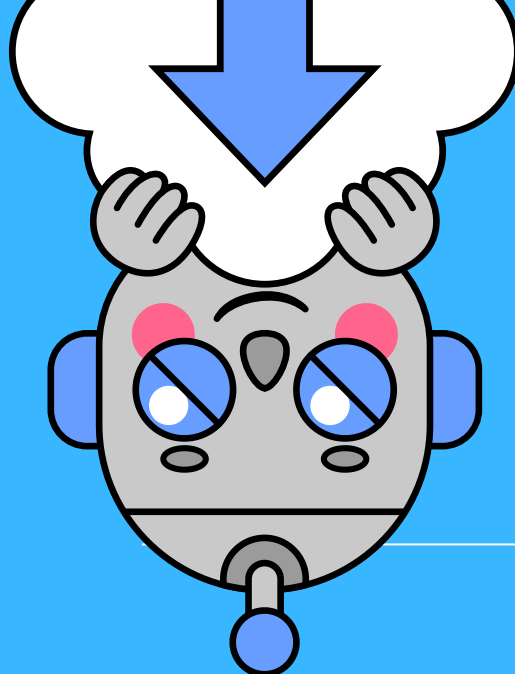


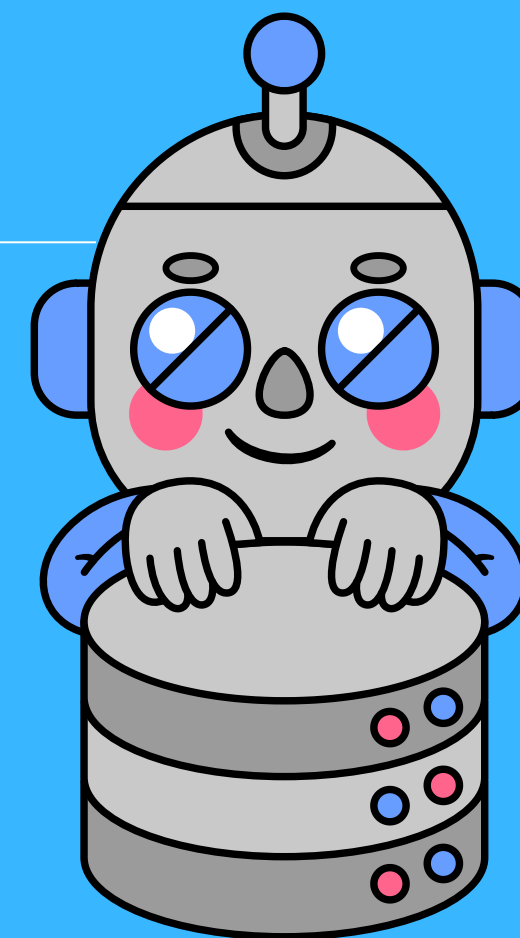
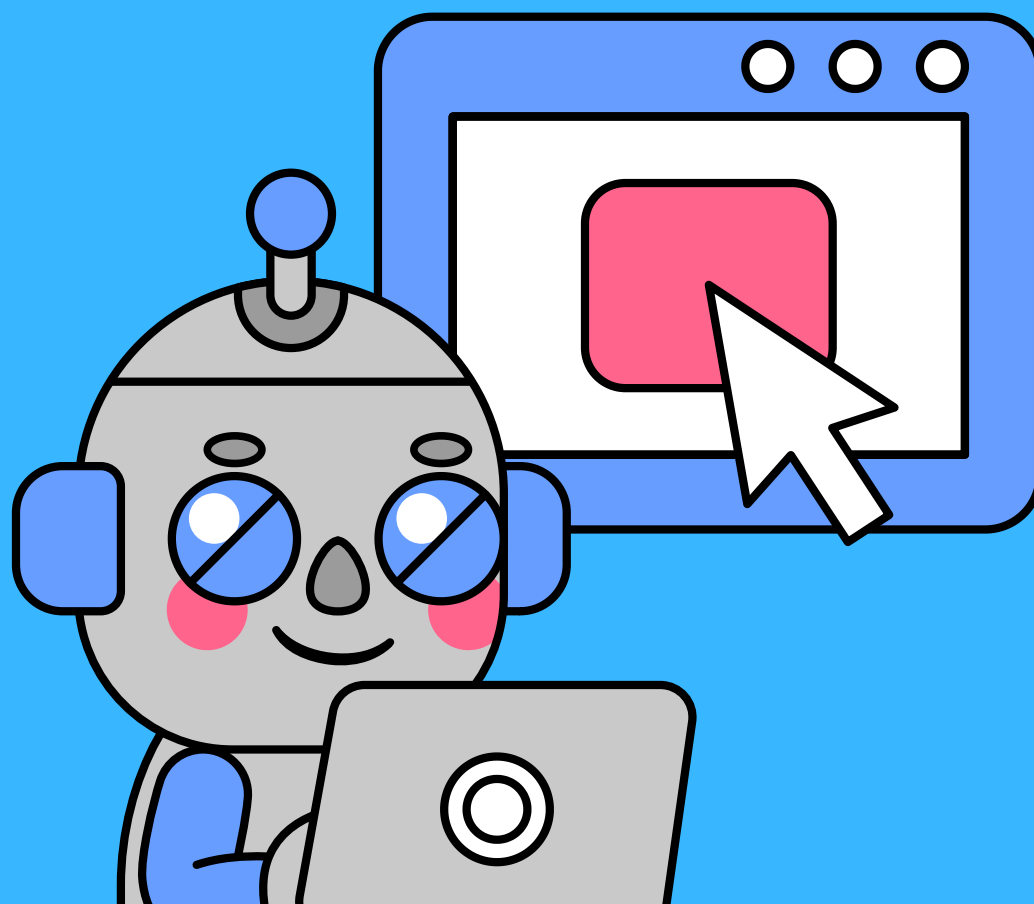


Artificial Intelligence

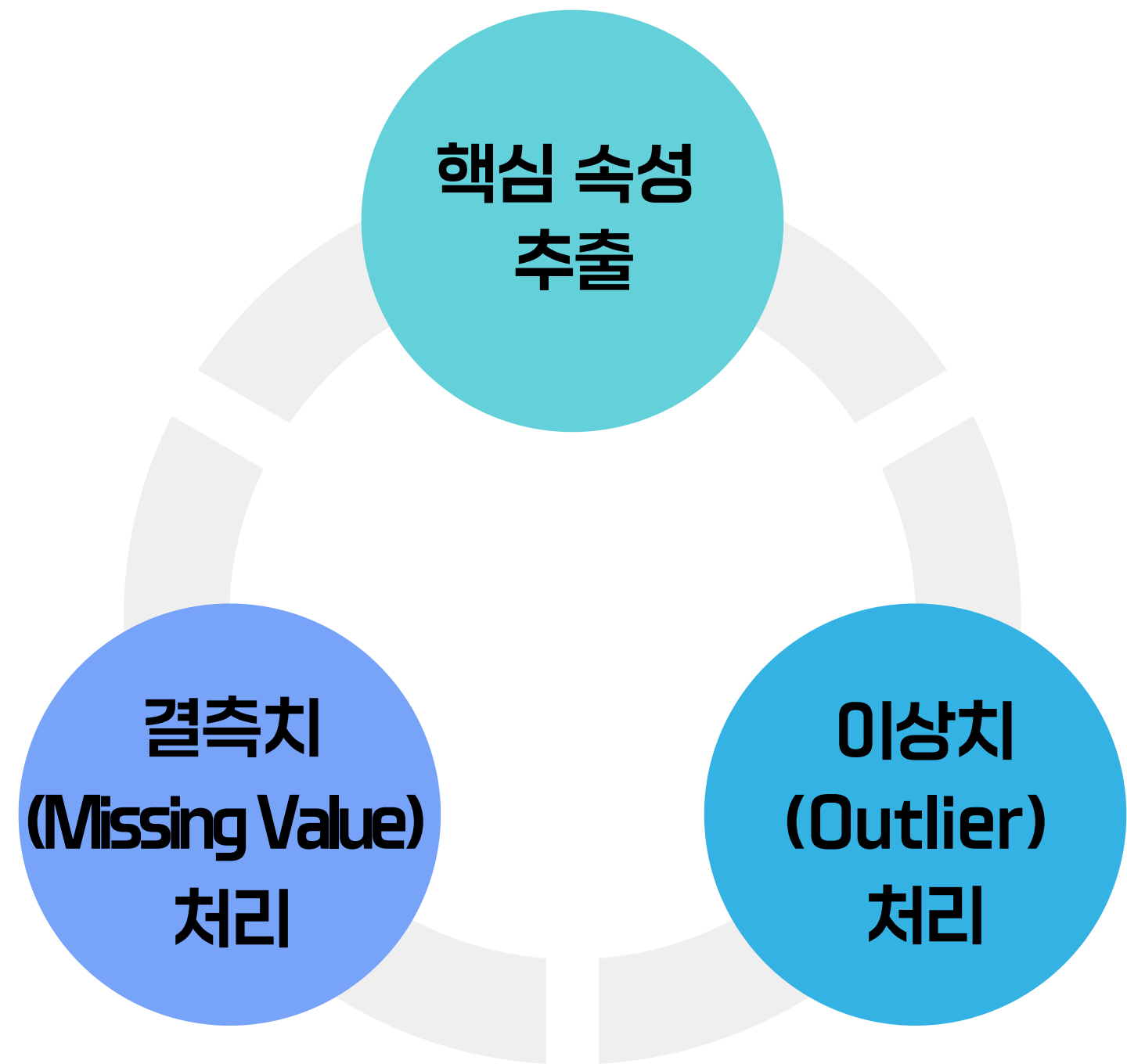
인공지능 기초

인공지능은 어떻게 스스로
똑똑해질까?

Chapter 03. 기계학습과 데이터(3)



1 데이터 전처리



1 이상치 처리하기

STATISTICAL CRITERION

사분위수 범위 (IQR) 기반의 이상치 감지 원리

단순히 느낌으로 "너무 크다", "너무 작다"를 판단하면 과학적이지 못합니다. 통계학에서는 전체 데이터를 크기 순서대로 정렬하여 4등분한 뒤, 가운데에 밀집한 핵심 데이터 구간을 기준으로 정상과 이상치의 수학적 경계선을 정의합니다.

제1사분위수 (Q1)

데이터를 순서대로 나열했을 때 하위 25%에 해당하는 값

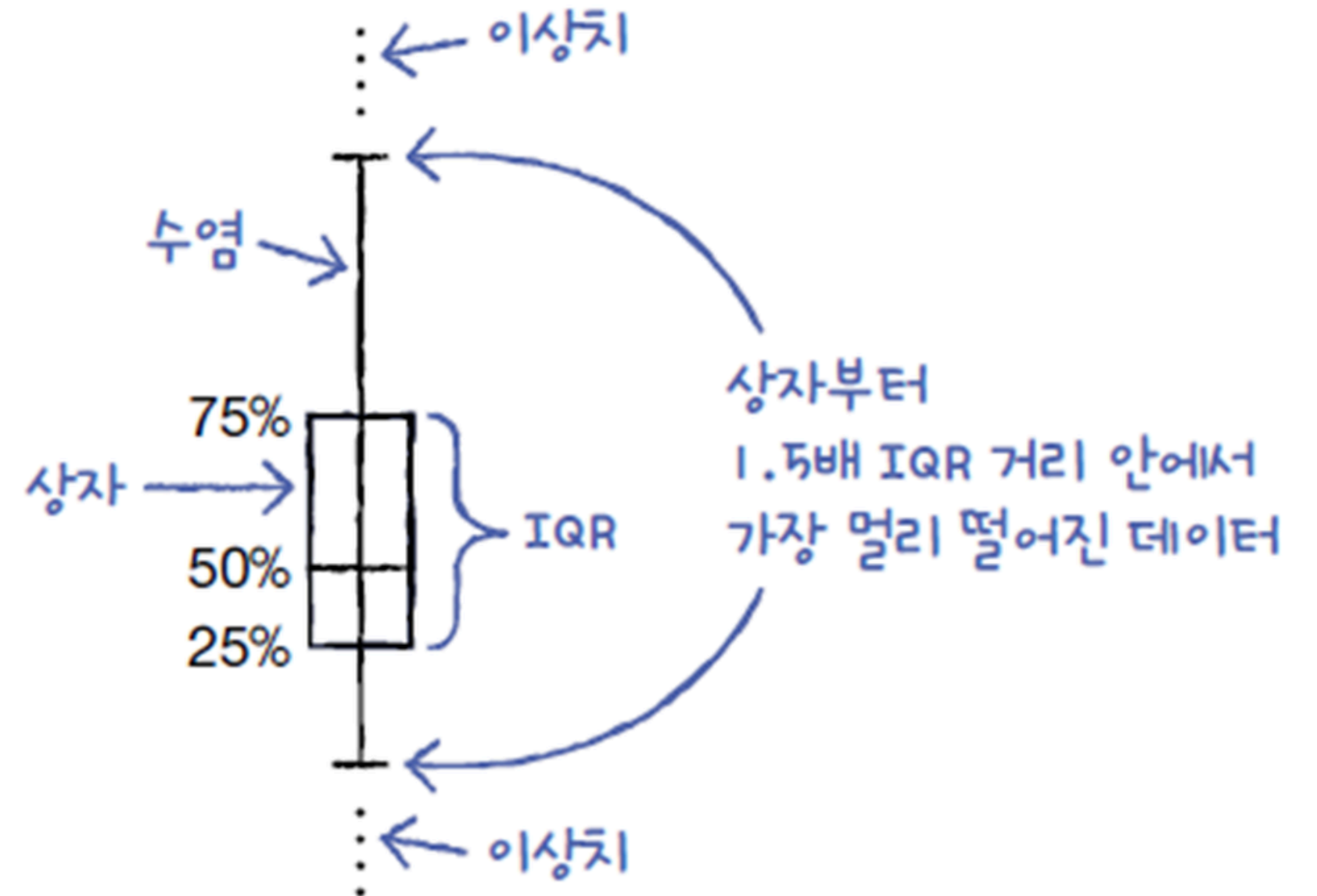
제3사분위수 (Q3)

데이터를 순서대로 나열했을 때 하위 75%(상위 25%)에 해당하는 값

사분위수 범위 (IQR, Interquartile Range)

$$IQR = Q3 - Q1$$

가운데 50%의 데이터가 모여 있는 순수 핵심 구간의 폭(길이)



1 이상치 처리하기

통계학적 기준에 따라 핵심 구간 폭(IQR)의 **1.5배**를 상한 조건과 하한 조건의 한계 울타리로 설정합니다. 이 울타리를 벗어나는 데이터는 모두 **이상치(Outlier)**로 판정하고 그 위치를 검출해 낼 수 있습니다.

이상치 판단 하한선 (Lower Fence)

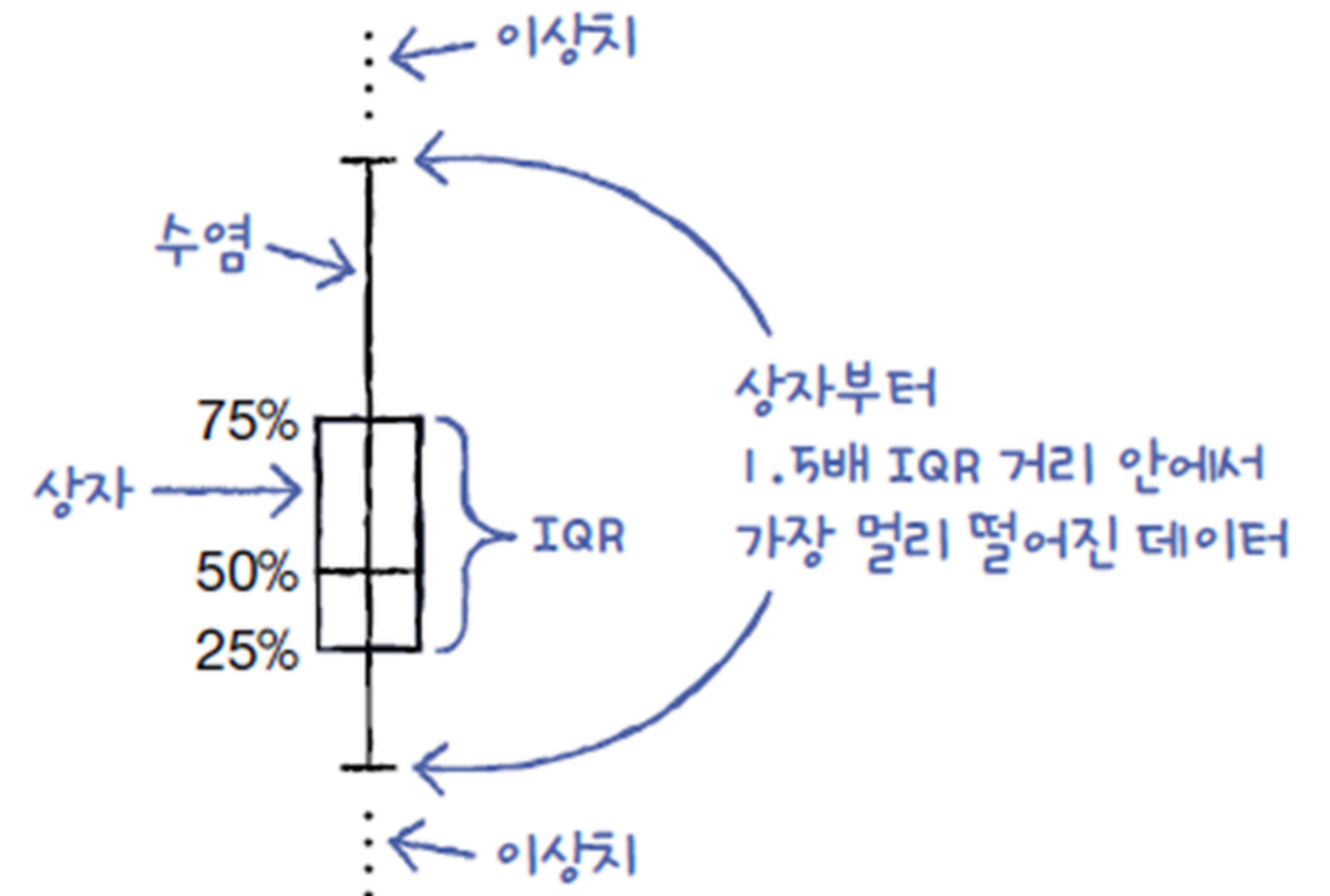
$$Q1 - 1.5 * IQR$$

이 연산 결과값보다 **작은 값**은 극단적으로 작은 이상치입니다.

이상치 판단 상한선 (Upper Fence)

$$Q3 + 1.5 * IQR$$

이 연산 결과값보다 **큰 값**은 극단적으로 큰 이상치입니다.



1 이상치 처리하기

통계학적 기준에 따라 핵심 구간 폭(IQR)의 **1.5배**를 상한 조건과 하한 조건의 한계 울타리로 설정합니다. 이 울타리를 벗어나는 데이터는 모두 **이상치(Outlier)**로 판정하고 그 위치를 검출해 낼 수 있습니다.

이상치 판단 하한선 (Lower Fence)

$$Q1 - 1.5 * IQR$$

이 연산 결과값보다 **작은 값**은 극단적으로 작은 이상치입니다.

이상치 판단 상한선 (Upper Fence)

$$Q3 + 1.5 * IQR$$

이 연산 결과값보다 **큰 값**은 극단적으로 큰 이상치입니다.

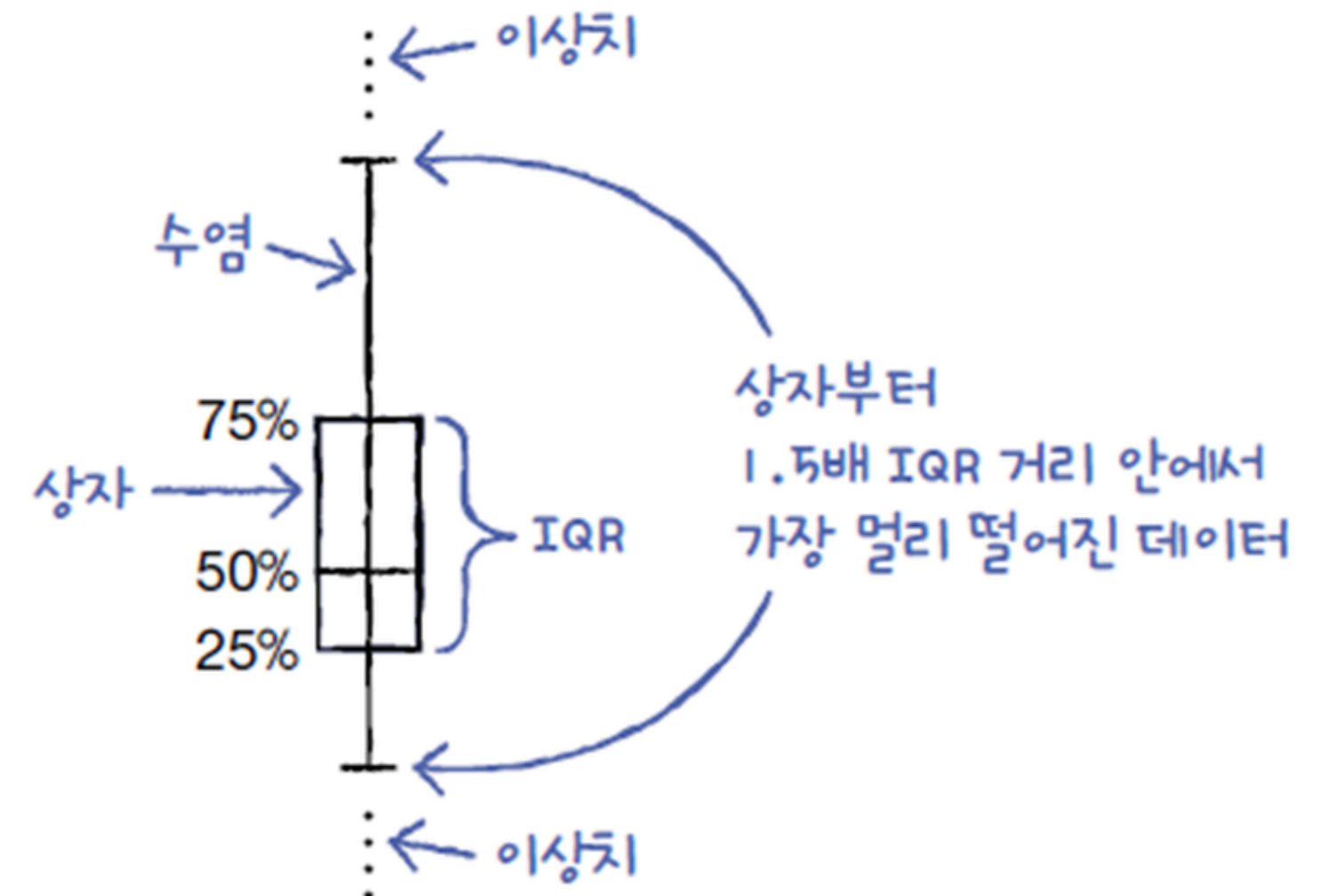
1) 데이터 프레임에서 Q1, Q3값 구하기 (quantile 활용)

$$Q1 = df['체중'].quantile(0.25)$$

$$Q3 = df['체중'].quantile(0.75)$$

2) IQR값 구하기

$$IQR = Q3 - Q1$$



1 이상치 처리하기

통계학적 기준에 따라 핵심 구간 폭(IQR)의 **1.5배**를 상한 조건과 하한 조건의 한계 울타리로 설정합니다. 이 울타리를 벗어나는 데이터는 모두 **이상치(Outlier)**로 판정하고 그 위치를 검출해 낼 수 있습니다.

이상치 판단 하한선 (Lower Fence)

$$Q1 - 1.5 * IQR$$

이 연산 결과값보다 **작은 값**은 극단적으로 작은 이상치입니다.

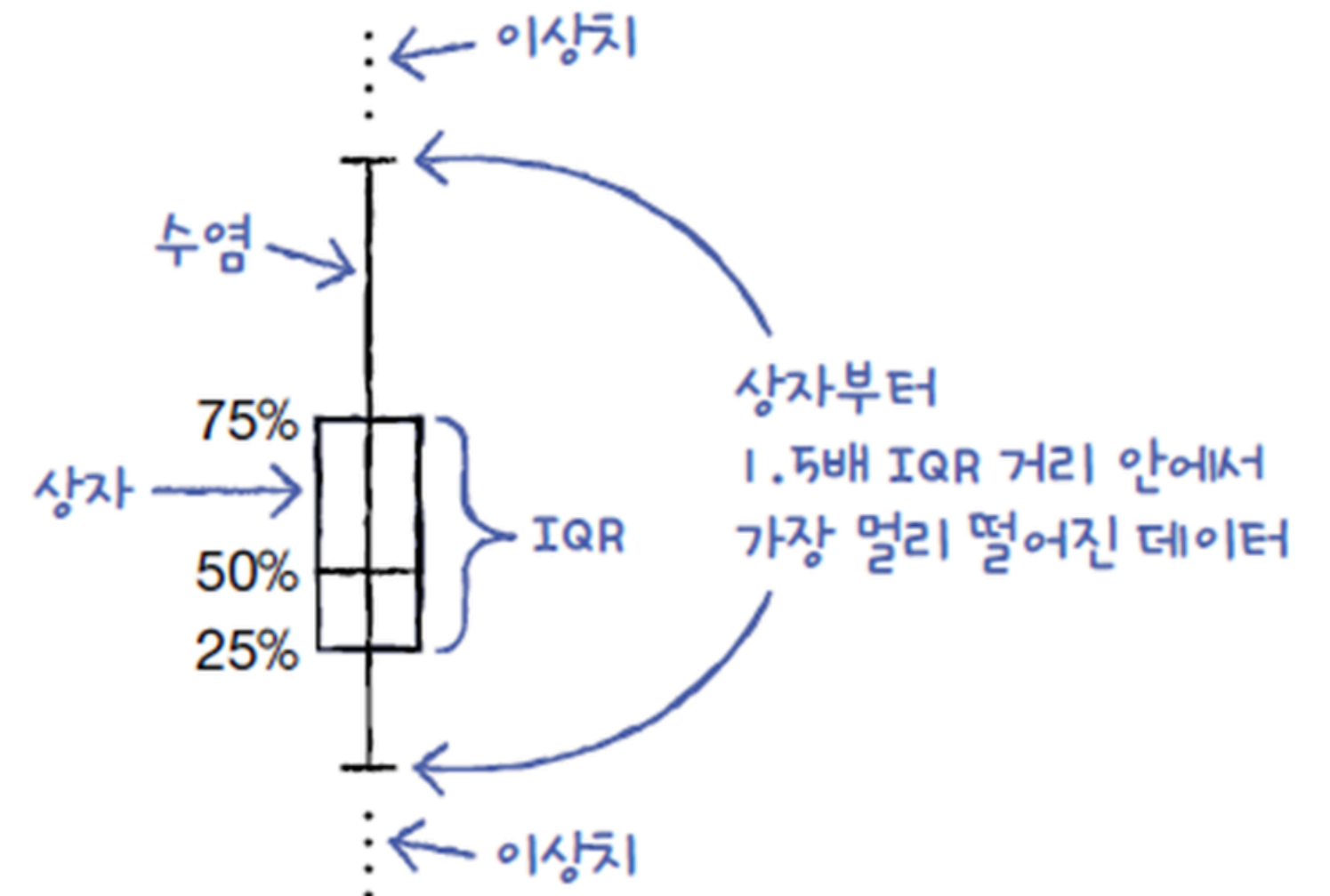
이상치 판단 상한선 (Upper Fence)

$$Q3 + 1.5 * IQR$$

이 연산 결과값보다 **큰 값**은 극단적으로 큰 이상치입니다.

3) 정상 범위만 추출하기

```
df2=df.query('체중>= (@Q1 - @IQR*1.5) and
체중<= (@Q3 - @IQR*1.5)')
```



2 데이터 형식 변경

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   주문일   10 non-null    object  
 1   지역     10 non-null    object  
 2   판매원   10 non-null    object  
 3   상품명   10 non-null    object  
 4   수량     10 non-null    float64  
 5   단가     9 non-null     float64  
 6   할인율   8 non-null     float64  
dtypes: float64(3), object(4)
memory usage: 748.0+ bytes
```

**** 날짜는 날짜 형식으로 바꾸어야 날짜에 따른 연산이 가능함**

날짜 형식: datetime

```
df['주문일'] = pd.to_datetime(df['주문일'])
```

3 속성명 변경하기

	주문일	지역	판매원	상품명	수량	단가	할인율
0	2025-10-01	서울	김사원	A	10.0	1000.0	0.10
1	2025-10-01	부산	이사원	B	5.0	2000.0	0.05
2	2025-10-02	서울	김사원	A	8.0	1000.0	0.10
3	2025-10-02	서울	박사원	C	3.0	3000.0	NaN
4	2025-10-03	부산	이사원	B	7.0	2000.0	0.05
5	2025-10-03	광주	김사원	A	5.0	NaN	0.10
6	2025-10-04	서울	박사원	C	999.0	3000.0	0.20
7	2025-10-04	부산	이사원	B	10.0	2000.0	NaN
8	2025-10-01	서울	김사원	A	10.0	1000.0	0.10
9	2025-10-05	인천	정대리	B	2.0	2000.0	0.05
10	NaT	NaN	NaN	NaN	NaN	NaN	NaN

****속성 이름 변경하기(rename)**

```
df=df.rename(columns={"단가":"가격","상품명":"제품"})
```

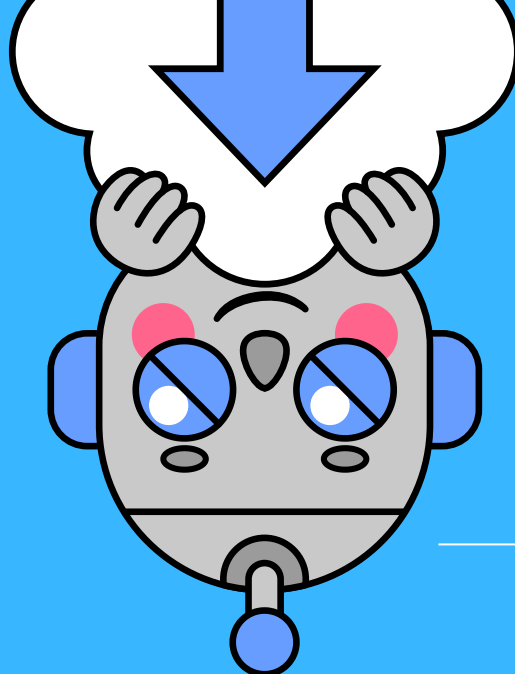

날짜를 인덱스로 설정하기

	주문일	지역	판매원	상품명	수량	단가	할인율
0	2025-10-01	서울	김사원	A	10.0	1000.0	0.10
1	2025-10-01	부산	이사원	B	5.0	2000.0	0.05
2	2025-10-02	서울	김사원	A	8.0	1000.0	0.10
3	2025-10-02	서울	박사원	C	3.0	3000.0	NaN
4	2025-10-03	부산	이사원	B	7.0	2000.0	0.05
5	2025-10-03	광주	김사원	A	5.0	NaN	0.10
6	2025-10-04	서울	박사원	C	999.0	3000.0	0.20
7	2025-10-04	부산	이사원	B	10.0	2000.0	NaN
8	2025-10-01	서울	김사원	A	10.0	1000.0	0.10
9	2025-10-05	인천	정대리	B	2.0	2000.0	0.05
10	NaT	NaN	NaN	NaN	NaN	NaN	NaN

****날짜가 인덱스가 되면 날짜의 흐름에 따라 값 비교가 가능**

`df=df.set_index('주문일')`

	지역	판매원	상품명	수량	단가	할인율
주문일						
2025-10-01	서울	김사원	A	10.0	1000.0	0.10
2025-10-01	부산	이사원	B	5.0	2000.0	0.05
2025-10-02	서울	김사원	A	8.0	1000.0	0.10
2025-10-02	서울	박사원	C	3.0	3000.0	NaN
2025-10-03	부산	이사원	B	7.0	2000.0	0.05
2025-10-03	광주	김사원	A	5.0	NaN	0.10
2025-10-04	서울	박사원	C	999.0	3000.0	0.20
2025-10-04	부산	이사원	B	10.0	2000.0	NaN
2025-10-01	서울	김사원	A	10.0	1000.0	0.10
2025-10-05	인천	정대리	B	2.0	2000.0	0.05
NaT	NaN	NaN	NaN	NaN	NaN	NaN



Artificial Intelligence

NEXT

인공지능과 학습(4) - 데이터 시각화

